

Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Carthage
Institut supérieur des technologies de l'information et de la
communication

Chapitre 3 : Les structures itératives

Plan

1. Introduction
2. Structure itérative Pour
3. Structure itérative TantQue
4. Structure conditionnelle do...while
5. Exercices

1. Introduction

Introduction du concept :

Exemple : Écrire un algorithme qui affiche 100 fois un message.

1ère Solution :

Algorithme Affichage

Début

Écrire ("Attention ")

Écrire ("Attention ")

...

Écrire ("Attention ")

Fin

⇒ Solution correcte mais programme trop long.

2ème Solution : Utilisation d'une structure itérative

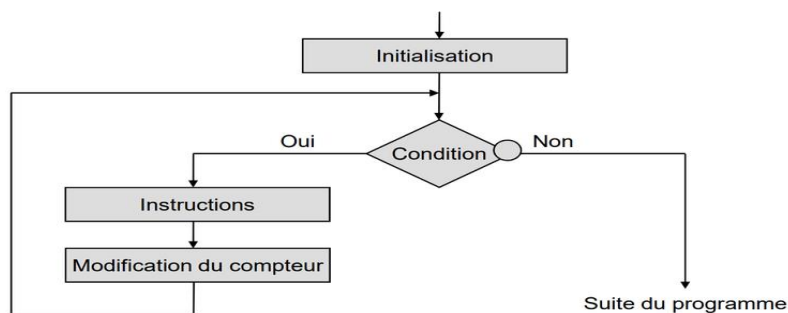
Définition :

Une structure itérative permet de répéter l'exécution d'une instruction ou d'une suite d'instructions plusieurs fois.

2. Structure itérative Pour (for en langage C)

Définition : Cette structure permet de répéter l'exécution des d'instructions selon un nombre connu de fois.

Organigramme



Syntaxe en algorithmique

Pour V de Vi à Vf Faire

<suite_instructions>

FinPour

Avec

V : Variable de contrôle, ou compteur

Vi : Valeur initiale de la variable v

Vf : Valeur finale de la variable v

Remarque

- Si $V_i < V_f$: Parcours Croissant de la variable v (pas de 1).
- Si $V_i > V_f$: Parcours Décroissant de la variable v de (pas de -1).
- FinPour permet d'incrémenter automatiquement le compteur.

Syntaxe en langage C

```
for(intialisation; condition; modification)
{<suite_d'instruction>;
}
```

La boucle pour permet de répéter l'exécution d'un bloc en automatisant les phases d'initialisation et de modification de la variable de boucle. En effet, le mot-clé for, qui précède le bloc, est suivi de trois expressions séparées par des points-virgules :

- La première permet d'effectuer l'initialisation : elle est exécutée une seule fois, au début de la boucle.
- La deuxième est la condition : elle est évaluée au début de chaque itération.
- La troisième est la modification : elle est exécutée à la fin de chaque itération.

Exemple 1 : Afficher les 10 premiers entiers à l'écran.

```
#include <stdio.h>
void main()
{
for(int i=1; i<=10; i++ )
{
    printf("%d", i);
}
}
```

Exemple 2 : somme des 10 premiers entiers à l'écran

```
#include <stdio.h>
void main()
{ int S=0;
for(int i=1; i<=10; i++ )
{ S=S+i ;
}
printf("somme des 10 premier entier est %d", S) ;
}
```

Remarque

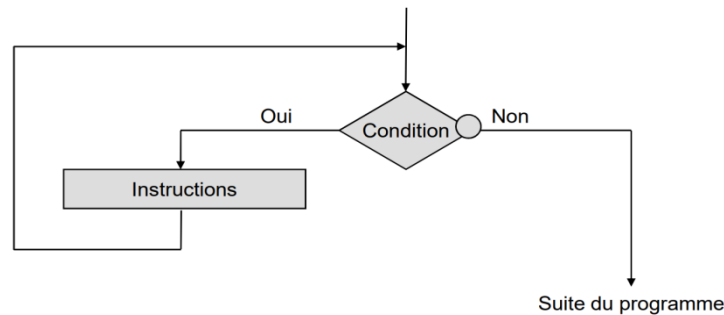
À noter qu'il est possible d'initialiser/modifier plusieurs variables de boucle en même temps en utilisant des virgules dans les expressions :

```
for(x=1,y=10; x=y; x++,y--)
```

3. Structure itérative TantQue (while en langage C)

Définition : Cette structure permet de répéter l'exécution d'une suite d'instructions dont le nombre d'itérations n'est pas connu à l'avance.

Organigramme



Syntaxe en algorithmique

```
TantQue <Condition> Faire  
<suite_d'instructions>  
FinTantQue
```

Syntaxe en langage C

```
while (condition)  
{  
<suite_d'instructions> ;  
}
```

Explication :

- Cette structure permet de répéter une suite d'instructions tant que la condition est satisfaite (c'est-à-dire que la condition prend la valeur true).
- La condition est testée avant l'exécution des instructions. Par conséquent, ces instructions peuvent alors ne pas être exécutées lorsque la condition n'est pas vérifiée (c'est-à-dire que la condition prend la valeur false).
- La condition doit forcément porter sur la variable de boucle. Celle-ci doit impérativement être initialisée avant le `while`. De plus, cette variable doit forcément être modifiée dans le bloc d'instructions.

Exemple1 : Lecture d'une suite de N valeurs dont le dernier est 0.

Algorithme Valeurs

```
Variables  
X : Entier  
Début  
  X ← 1  
TantQue (X <> 0) Faire  
  Écrire (" Donner une valeur ")  
  Lire (X)  
FinTantQue  
Écrire ('Fin de Saisie')  
Fin
```

```
#include <stdio.h>  
void main()  
{  
  int x=1;  
  while (x!=0)  
  {  
    printf("donner un entier  
    différent de zéro");  
    scanf("%d", &x);  
  }  
  printf("fin de saisi");  
}
```

Exemple 2 : Écrire un algorithme qui convertit un nombre de minutes en un nombre d'heures et de minutes (sans utiliser DIV et MOD). Ex : 320 mn = 5h 20 mn

Algorithme Conversion

```
Variables
NB, H : Entier
Début
Écrire (" Donner un nombre ")
Lire (NB)
H = 0
TantQue (NB>=60) Faire
  NB = NB - 60
  H = H + 1
FinTantQue
Écrire (" Le nombre en heures = ", H,
" et le nombres en minutes = ", NB)
Fin
```

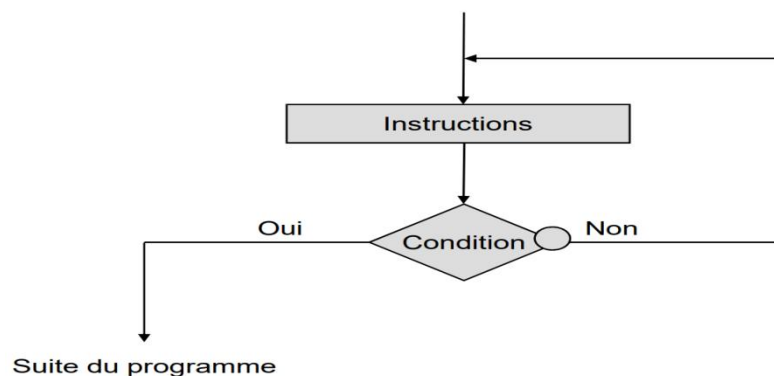
```
#include <stdio.h>
void main()
{
  int NB, H;
  printf("donner un nombre");
  scanf("%d", &NB);
  H=0;
  while (NB>=60)
  {
    NB=NB-60;
    H=H+1;
  }

  printf("nombre d'heure et de
minute sont %d %d",H, NB);
}
```

4. Structure itérative Répéter (do....while en langage C)

Définition : Cette structure permet de répéter l'exécution d'une suite d'instructions dont le nombre d'itérations n'est pas connu à l'avance.

Organigramme



Syntaxe en algorithmique

Répéter
<suite_d'instructions>
Jusqu'à <condition>

Traduction en langage C

```
do
{
<suite_d'instructions> ;
}while !(condition) ;
```

Explication

- Cette structure permet de répéter une suite d'instruction jusqu'à ce que la condition soit vraie.
- La condition est exprimée sous la forme d'une expression entre parenthèses, placée après un `while`, comme pour la boucle tant que. La différence ici, est que le `while` est placé après le bloc, et qu'elle est suivie d'un point-virgule (i.e. ;). N'oubliez surtout pas ce point-virgule.
- La condition n'est testée qu'après une première exécution des instructions $\Rightarrow$ par conséquent, ces instructions doivent être exécutées au moins une fois.

Exemple : Lecture d'une valeur strictement positive.

Algorithme Valeur

Variables

X : Entier

Début

Répéter

 Écrire ("Donner une valeur")

 Lire (X)

JusQu'à (X > 0)

Fin

```
#include <stdio.h>
void main()
{
    int N;
    do
    {
        printf("donner une valeur");
        scanf("%d", &N);
    }while (N<=0);
}
```

Exercice 1

Ecrire un programme en C qui permet de :

- Afficher les diviseurs d'un entier X donné (autre que 1 et lui-même).
- Et calculer et afficher le nombre de diviseur (Lorsque X n'a pas de diviseurs alors on fera apparaître le message 'pas de diviseur')

Corrigé exercice 1

```
#include <stdio.h>
void main()
{
    int N, i, nb, m;
    printf("donner un entier : ");
    scanf("%d", &N);
    nb=0;
    m=N/2;
    for (i=2; i<=m; i++)
    {
        if (N % i==0)
        {
            printf("%d ", i);
            nb=nb+1;
        }
    }
    if (nb == 0)
        printf("pas de diviseur c'est un entier premier");
    else
        printf("\nle nombre de diviseur de %d autre que 1 et lui même est %d:", N, nb);
}
```

Exercice 2

Ecrire un programme qui saisit un entier et qui l'affiche à l'envers. Par exemple, l'utilisateur saisit 123456 et le programme affiche 654321. Pour cela il faudra utiliser la division et le modulo.

Corrigé exercice 2

```
#include <stdio.h>
void main()
{
    int N, rest;
    printf("donner un entier : ");
    scanf("%d", &N);
    while(N!=0)
    {
```

```

rest=N%10;
printf("%d", rest);
N=N/10;
}
}

```

Exercice 3

Ecrire un programme qui permet de saisir un entier strictement positif N et d'afficher s'il est premier ou non (en utilisant la boucle Tantque et une variable logique). Un nombre est dit premier s'il est divisible uniquement par 1 et par lui-même.

Corrigé exercice 3

```

#include <stdio.h>
#include <stdbool.h>
void main()
{
    int N, i, m;
    bool P;
    do{
        printf("donner un entier : ");
        scanf("%d", &N);
    } while (N<=0);
    i=2;
    P=true;
    m=N/2;
    while ((i<=m) && (P==true))
    {
        if (N%i==0)
            P=false;
        i++;
    }
    if (P==true)
        printf("%d est premier", N);
    else
        printf("%d n'est pas premier", N);
}

```

Autre solution

```

#include <stdio.h>
void main()
{
    int N, i, m;
    do{
        printf("donner un entier : ");
        scanf("%d", &N);
    } while (N<=0);
}

```

```

i=2;

m=N/2;
while ((i<=m) && (N%i!=0))
{
    i++;
}
if(i<=m)
printf("%d n'est pas premier", N);
else
printf("%d est premier", N);
}

```

Exercice 4

Ecrire un programme qui permet de vérifier si un entier donné est un nombre d'Armstrong (c'est-à-dire un nombre cubique). Un entier est dit cubique s'il est égal à la somme des puissances cubiques de ses chiffres. Exemple, $153 = 1^3 + 5^3 + 3^3$ est un nombre cubique.

Corrigé exercice 4

```

#include<stdio.h>
void main()
{
    int nbr, a, y, s=0;

    printf(" Entrez un nombre: ");
    scanf("%d", &nbr);

    y=nbr;

    while (nbr!=0)
    {
        a=nbr%10;
        s=s+(a*a*a);
        nbr=nbr/10;
    }
    if(y==s)
        printf(" %d est un nombre Armstrong",y);
    else
        printf(" %d n'est pas un nombre Armstrong",y);
}

```